

Castia

engineering dossier

BRANCH · CLAUDE/AURA-ARCHITECTURE-DESIGN-8VMCYN
145 COMMITS · 828 TESTS GREEN
STATUS · PRE-LAUNCH, UNMEASURED

Castia adapts song lyrics and comic dialogue across six languages so the **feeling** survives, not just the words. This document covers what exists, how it works, what is proven, what is merely built, and what to do next. It is written to be accurate rather than flattering: the largest single finding is that the project's own measurement apparatus has never been run.

CONTENTS

00	The recommendation	05	The comics pipeline
01	What Castia is	06	System architecture
02	The Writers' Room	07	The web product
03	Song DNA	08	Evidence & tiers
04	The verifier	09	Known gaps
		10	How to improve it
		11	Risks

Stop building. Start measuring.

You asked what I suggest. This is the whole answer, and everything after it is supporting detail.

Castia has a **145-commit architecture and zero measurements**. The engine is elaborate, well-tested at the code level, and honest in its documentation. But not one of its quality claims has been checked against a real alternative on real material.

THE FINDING THAT MATTERS

A complete blind-benchmark harness already exists in `benchmark/` — a runner, output normalisation, reviewer blinding, and significance testing. It compares Castia against a single GPT prompt, a single Claude prompt, Google Translate, and published human translations.

It has never been run. There is no `benchmark/runs/` directory, no `benchmark/manual/`, and `benchmark/corpora/` contains a single notes file. Meanwhile 27 separate capability entries are marked *Tier 0 — unmeasured*.

This matters more than any feature, because it is the thing that decides whether the feature work was worth doing. The single-GPT-prompt comparison is the crucial one: it isolates Castia's *process* — the Translator/Adapter/Judge room, the constitution, the verifier — from the raw model underneath. If a well-written single prompt scores the same, the entire architecture is expensive ceremony. If it doesn't, you have the strongest possible claim in the category, and it is defensible with numbers.

Your own reaction to *Jiya Jale* — "alright, but not something I'd pay for" — is currently the only quality signal the project has. That is one data point, from one listener, on one song. It produced two real engine fixes. Ten songs and three reviewers would produce a roadmap.

The order I would take it in

1 Re-run Jiya Jale

Costs nothing, takes minutes, and directly tests the two fixes shipped this session — the hollow-deviation check and the prose-collapse escalation. Either the verse comes back with its line structure intact and no comma-sized "improvements," or it doesn't. That single result tells you whether the corrective pass actually works in production or only in tests.

Effort: minutes · Needs: nothing you don't have

2 Run the benchmark you already built

Six songs already sit in `examples/`. Stage one needs only an API key: `python -m benchmark.cli run --corpus examples --run-id pilot_1`. Stage two blinds the outputs. Stage three needs two or three bilingual reviewers — the genuine bottleneck, and worth solving now rather than after another architecture round.

Effort: days, mostly recruiting reviewers · Converts ~27 Tier 0 claims into evidence

3 Cross-panel speaker reconciliation

The one substantial feature I would still build before measuring, because it needs no new infrastructure and closes a loop that is currently half-open. The vision pass already emits panel-local `Character_A` labels plus a `speaker_appearance` descriptor; nothing yet merges those into real cast members across a chapter. Until it does, per-character voice consistency — the thing that separates a localisation tool from a translation toy — cannot actually happen.

Effort: 1–2 days · Needs: no services, no weights

4 Stand up exactly one ML service

Three service boundaries are built and none has met a real model. Pick **inpainting** first: its output is judged by eye in seconds, it needs no reviewer panel, and the quality delta is the most visible of the three. If LaMa's reconstruction on real artwork is obviously better than the OpenCV smear, that justifies the infrastructure for the other two.

Effort: deployment work · Needs: a GPU box or a hosted IOPaint

What I would not do next: add more capability. Every feature added before measurement widens the unmeasured surface and increases the cost of discovering the premise was wrong. The architecture is far enough ahead of the evidence that more architecture is now the risky choice.

A rewrite that still means it

The product thesis, the problem it attacks, and the shape of the thing as it exists today.

Machine translation optimises for semantic equivalence: it asks "what do these words mean?" and returns the closest words in another language. For prose that is usually right. For a lyric it is usually wrong, because a lyric's meaning is carried as much by rhythm, register, idiom, repetition and restraint as by denotation. A literal rendering can be perfectly accurate and completely dead.

Castia's premise is that adaptation — the thing a human translator of songs actually does — is a *separable, auditable process*, not an unexplainable act of taste. Concretely: produce a literal anchor, produce several genuine alternatives, then have a judge rule between them against that anchor and **write down why** each departure was earned.

The output is a three-part artefact for every section, which is also the product's signature UI:

LITERAL	CASTIA
Pause for a moment, let my heart settle. How can I stop you? Every sorrow coming my way slips away as I fill my eyes with you.	Stay — one breath, steady this heart. How do I hold you here? Every grief that finds me, gone. Eyes full of you.
WHY IT CHANGED <i>The first version reads like a sentence someone translated. This one reads like something someone would actually say — and it says the same plea twice, the way she really does, instead of smoothing it into two different lines.</i>	

That third panel is the product. Any tool can return different words; the claim Castia makes is that every difference is deliberate, logged, and defensible. The verifier described in §04 exists to make that claim mechanically enforceable rather than aspirational.

Scope as it stands

DIMENSION	CURRENT STATE
Languages	English, Hindi, Japanese, Korean, Spanish, Urdu — any direction, 30 ordered pairs
Music input	Pasted lyrics, or YouTube captions imported as a reviewable draft
Comics input	Panel images; OCR pre-fills a per-panel script the human reviews • BETA
Output	Literal / adapted / why per section, shareable link, CSV export for comics
Accounts	Google sign-in, saved history, favourites, collections
Public API	POST /v1/adapt , POST /v1/comics/adapt , key-authenticated • BETA
Billing	Not live. Pricing page states this plainly rather than implying otherwise

The project by the numbers

ENGINE + SERVER 12,108 lines of Python	TEST CODE 10,831 lines · 0.89 ratio	TESTS GREEN 828 742 py · 86 vitest	ENGINE MODULES 30 python files	FRONTEND 100 ts/tsx files
DOCUMENTATION 8,418 lines across 11 docs	UNMEASURED 27 Tier 0 claims	BENCHMARKS RUN 0 harness idle		

A test-to-source ratio near 1:1 and 8,400 lines of documentation are unusual at this stage and reflect a genuine discipline in the codebase: capabilities are recorded with their limits, and claims are tiered by how well they are evidenced. That same discipline is what makes the final row of the table above so conspicuous.

Three roles, one binding ruling

The core mechanism. Every line of every section, music or comics, passes through this.

STAGE 1

Translator

One literal anchor translation.
Never shipped as output — it is the floor everything else is measured against.

STAGE 2

Creative Adapter

Five candidates, one per adaptation philosophy — maximum fidelity, native lyricist, performance-first, emotion-first, genre-first.

STAGE 3

Judge

Rules between them against the anchor, logs every real departure with a specific reason, and may consult specialists first.

The anchor is the load-bearing idea. Without it, a "creative translation" has nothing to be creative *relative to*, and there is no way to distinguish an earned departure from a hallucination. With it, every difference between the shipped line and the literal reading is a measurable delta that must be justified in a ledger entry.

Specialists

The Judge triages first. If it can rule confidently it does so immediately; if not, it may summon up to three consultants before ruling. They exist to arbitrate contested ledger entries, not to give generic critique:

- **Cultural historian** — whether a culturally-loaded reference survives direct rendering or needs re-expression.
- **Native speaker** — authenticity veto. A candidate called a failure here is disqualified regardless of other merit.
- **Psychologist** — whether the emotional register of a candidate matches what the source actually does.

This is a real cost decision: triage-and-rule is one model call, the specialist path is up to five. The Judge decides which it needs, guided by precomputed routing signals it may override but must not ignore silently.

The constitution

The room operates under a written set of laws, enforced downstream by the deterministic verifier (§04). The essential ones:

LAW	WHAT IT FORBIDS
Law 1 — No Invention	Any change to the anchor that isn't logged in the deviation ledger with a specific reason.
Law 1 — Burden of Change	The inverse: claiming a change that wasn't actually made. Added this session after a real failure.
Law 3 — Compression Floor	Merging lyric lines into running prose, or dropping repeated lines the source deliberately repeats.
Law 4 — Restraint Ceiling	Naming an emotion the source only implies, adding intensifiers, or explaining what was left implicit.

DESIGN PRINCIPLE

The laws are two-sided by construction. Most guardrails only catch a model doing *too much*. Castia's also catch it doing too little — shipping a translation and calling it an adaptation. A system that only punishes over-writing scores a pure literal translation as perfect.

The corrective pass

After a full song is adapted it is verified, and any error-severity finding triggers exactly one corrective attempt on that section. Two routes exist, chosen per section:

- **Re-judge** the existing candidates with the finding as context. Correct for most errors, which are about the Judge's *pick* rather than the candidate pool.
- **Regenerate** a fresh candidate pool with explicit feedback. Necessary when every existing candidate already lost the thing the finding is about — no amount of re-judging recovers a repeat or a line structure that nothing in the pool has.

The pass is capped at one round and does not re-verify itself. That bound is deliberate: an uncapped loop risks oscillation, where each fix introduces a different violation, with no guaranteed termination.

Analysing the work before touching it

One analysis pass per song, before any adaptation, producing the context every later stage reads.

Adapting a section in isolation produces locally plausible lines that don't cohere. Song DNA runs once over the whole text and captures what the song is *doing*, so each section is adapted with knowledge of the whole:

- **Artistic thesis and arc shape** — what the song is about and how it moves.
- **Motifs** — images that recur, with their occurrences, so a motif renders consistently rather than being re-invented each time it appears.
- **Symbols**, classified as archetypal, cultural, or personal-to-the-work — because each class needs a different strategy. An archetypal symbol often survives direct rendering; a culturally-specific one needs adaptation; one established earlier in this very song must be rendered consistently with its own prior occurrences.
- **Ambiguities** the songwriter left deliberately open, which must stay open rather than being resolved by a translator's confidence.
- **Per-section narrative function**, and for comics a per-character voice profile.

Room Memory then carries decisions forward: once a motif is rendered a certain way, later sections inherit that decision rather than re-deciding. This is what makes a whole song read as one work.

Language profiles

Each supported language carries a profile with real, script-specific machinery rather than a single generic prompt: grapheme-to-phoneme conversion for Hindi (including a schwa-deletion heuristic) and Urdu, syllable counting grounded in the actual script, and a constitution block appended to prompts for that language. Syllable counts handed to the Judge are computed deterministically, not guessed by the model.

The part that isn't a model

`engine/verify.py` — 1,248 lines of deterministic checking, and the most distinctive thing in the codebase.

Everything else in the engine is a model doing something clever. The verifier is arithmetic. It takes a finished ruling and mechanically checks it against the constitution, producing findings at warning or error severity. Only errors trigger a corrective retry — warnings were designed with disclosed false-positive risk and deliberately do not auto-rewrite anything.

WHAT IT ACTUALLY MEASURES

CHECK	MECHANISM
Ledger coverage	Token-level diff between anchor and shipped line; every changed word must appear in some deviation entry.
Ledger integrity	Every deviation's quoted fragments must actually appear in the anchor and the final line. A fabricated audit trail is an error.
Adaptation distance	0.0 when the shipped line is the anchor verbatim, rising as it departs. The counterweight that stops a pure translation scoring perfectly.
Justification quality	Pattern-matches vacuous reasons — "flows better", "more natural", "clearer rhythm and flow".
Burden of Change	Fires when distance is near zero <i>and</i> a logged reason is vacuous. Both conditions, so a small-but-real edit with a specific reason passes.
Line collapse	Anchor line count vs shipped line count — catches a verse flattened into prose.
Repeat preservation	Detects a refrain or repeated couplet the source states twice and the output states once.
Completeness	Shipped line vs the median length of its own sibling candidates — catches truncation.
Restraint	Emotion words, intensifiers and explanatory connectives present in the output but absent from the anchor.
Rhyme & stress	Real phonetic analysis via CMU dictionary and per-language G2P. English-only for the dictionary-backed checks, and it declines honestly rather than reporting a number it can't compute.

WHY THIS IS THE DIFFERENTIATOR

Competitors can match Castia's model quality by using the same model. They cannot easily match a deterministic audit layer that catches the specific ways adaptation fails, because building it requires having decided what adaptation quality *is* in enough detail to compute it. That decision is the actual intellectual property here.

► THE TWO FAILURES FOUND AND FIXED THIS SESSION

Same room, harder inputs

Webtoons and comics reuse the Writers' Room unchanged. Everything novel is in getting text *into* it and pictures back out.

Comics were the right second medium precisely because the hard part — deciding what a line should become — was already solved. What comics add is a vision problem at the front and an image-editing problem at the back, with the same adaptation engine in the middle.

The read path, as rebuilt

Originally one Cloud Vision call did detection and recognition together, which forced everything downstream to wait for the slow half before it could even learn where the text was. It is now three components behind protocol boundaries:

STEP 1 · ALONE

Detector

Finds text regions and issues a `node_id` per box. Fast, cheap, and the join key for everything after it.

STEP 2 · PARALLEL

Recogniser

Reads those regions. Script-routed: a Japanese specialist where one is configured, a general recogniser elsewhere.

STEP 2 · PARALLEL

Narrative Director

A vision model reading the same boxes for text type, speaker, tone and lettering emphasis.

Because both readers are handed the same box ids, step 2's latency is the slower of the two rather than their sum, and their results merge **exactly** on `node_id` instead of being matched by text similarity. That distinction is not cosmetic: similarity matching fails precisely on the bubbles that need help most — garble a bubble badly enough and no score pairs it with the model's clean reading — and a mis-pairing sends the redraw step at the wrong artwork.

The Narrative Director schema

Per text node: verbatim source text; a type from *speech / thought / shout / whisper / narration / sfx / signage*; a panel-local speaker label with a visual appearance descriptor, a self-reported confidence and a visible/off-panel flag; a bounded tone plus a free-text note; lettering emphasis; and a reading index.

Two constraints in that schema are deliberate and worth defending. **Tone is bounded** because free-text emotion cannot be compared across panels and so could never drive consistency, only decorate a UI. **Speaker labels are panel-local** because a single-panel call cannot know its `Character_A` is the same person as six

panels ago — appearance is the join key a reconciliation step would use, and asserting chapter-wide identity from one panel would be fabrication.

Redraw and typesetting

An optional step erases the original lettering and typesets the adapted line back in. Inpainting sits behind a boundary with two implementations: a remote LaMa/IOPaint service, and local OpenCV Telea as the default and fallback.

Verified rather than asserted: a real panel run through this code erases text from a plain speech bubble invisibly, and leaves an obvious horizontal smear when the same erase crosses a striped background — the diagonal pattern simply stops. That is the exact case LaMa exists for, and it is now demonstrated rather than claimed.

Honest limits of the redraw as it stands: one bundled font for every line regardless of the original lettering; a heuristic text colour that assumes dark-on-light; rectangular boxes rather than true bubble shapes; and no layered export. The endpoint now reports which inpainter actually ran, so a silent fallback cannot masquerade as a LaMa reconstruction.

Three tiers, one engine

How the pieces are deployed and why they're split the way they are.

The Next.js frontend runs on Vercel. The Python engine runs as a persistent container on Railway with Postgres, because a full song is minutes of sequential model calls — far past a serverless function ceiling. The browser never talks to the engine directly; Next.js proxies every call, which is also where the shared internal secret is applied.

engine/	the adaptation engine – no HTTP, no database
pipeline.py	orchestration + the bounded corrective pass
writers_room_v1.py	Translator / Adapter / Judge / specialists
verify.py	1,248 lines of deterministic constitution checking
song_dna.py	whole-song analysis before adaptation
prompts.py	every prompt, in one auditable place
llm_client.py	provider adapters (OpenAI active, Anthropic kept)
language_profile.py, g2p_hi.py, g2p_ur.py, rhyme.py, rhythm.py	
comics_*.py	detect · recognize · vision · align · read · redraw · inpaint
server/	FastAPI wrapper: HTTP, auth, cache, quota, jobs
main.py	1,319 lines – every endpoint
accounts.py, api_keys.py, cache.py, db_models.py, jobs.py, quota.py	
web/	Next.js – 100 ts/tsx files, 17 routes, 19 API proxies
benchmark/	blind comparison harness – built, never run
tests/	10,831 lines
docs/	8,418 lines across 11 documents

The service-boundary pattern

Three heavy-ML components follow an identical shape, adopted deliberately: a Protocol, a remote implementation behind an environment flag, and a lightweight local fallback that keeps tests and local development free of infrastructure.

COMPONENT	REMOTE	FALLBACK	FLAG	STATE
Detector	comic-text-detector / YOLO	Cloud Vision boxes	CASTIA_TEXT_DETECTOR_URL	● UNTESTED
Recogniser	manga-ocr (Japanese only)	Cloud Vision	CASTIA_MANGA_OCR_URL	● UNTESTED
Inpainter	LaMa / IOPaint	OpenCV Telea	CASTIA_INPAINT_URL	● UNTESTED

The reason for the pattern is concrete: `torch` and `transformers` would take the API image from roughly two hundred megabytes to several gigabytes, plus resident memory for weights. Keeping them out of the main container is what lets the API stay small, scale independently, and keep working when a model service is down.

A CORRECTION WORTH RECORDING

The original plan was to replace Cloud Vision with manga-ocr outright. Its own package metadata reads *"OCR for Japanese manga"* — Japanese only. A straight swap would have upgraded one language and reduced the other five in the roster to nothing. Hence a registry routed by script rather than a replacement, which is also more extensible: adding a Korean specialist later is a registration, not a rewrite.

Operational behaviour

- **Content-addressed cache** — the same text returns the same result without re-running the engine, and the cache id doubles as the share-link slug.
- **Background jobs** — long songs run past the HTTP request in a Postgres-backed job table, so a poll landing on a different worker still finds the status.
- **Two quota dimensions** — per-IP for anonymous browser use, per-key for the public API, since many legitimate API calls share one IP.
- **Verification in production** — every web run is verified, corrected once if needed, then verified again purely for logging, so residual problems stay visible in stdout rather than only under a CLI flag.

Two mediums, one front door

17 routes, and the reasoning behind the ones that matter.

`/` is a neutral chooser between Music and Webtoons, presented as equals rather than flagship-plus-experiment. A returning visitor is redirected straight to their remembered medium, so the chooser is a one-time screen rather than a permanent toll gate. Webtoons carries a Beta badge because it genuinely lacks parity — no save, collections, or share links, and a synchronous adapt call that can time out on a long chapter.

`/music` is the main workflow: a persona toggle, language pickers, paste-or-YouTube input, and the real comparison screenshot brought up beside the headline so the page's one piece of proof is visible without scrolling. `/comics` is a panel-by-panel review workspace — upload, batch OCR, per-panel speaker and text fields, adapt, redraw, CSV export.

A rule the codebase keeps

NEVER OVERWRITE A HUMAN

OCR pre-fills an empty field and never replaces text a person has edited. The same applies to the speaker field, which is only auto-filled when exactly one speaker was identified for a panel — two speakers in one panel is a conversation, and picking either would attribute half the lines to the wrong character. Every automated fill is a draft a human reviews, never an answer.

Discoverability

Per-page metadata and canonicals, `robots.txt` with explicit rules for AI answer-engine crawlers, `sitemap.xml`, an `llms.txt` summary written for citation, and JSON-LD for Organization, SoftwareApplication and the FAQ. The structured data deliberately carries no fabricated ratings and no priced offer for the tier that isn't billed — only the real free tier.

What is actually known

The project's own honesty system, and the uncomfortable summary it produces.

`docs/CAPABILITY_MATRIX.md` — 213 KB, the largest document in the repo — records every capability with an evidence tier. This is a genuinely good practice and rare at this stage. It is also what makes the gap legible:

TIER	MEANS	EXAMPLES
• TIER 1	Deterministic and verified by tests	The verifier's checks, node_id alignment, mask geometry, caching, quotas, bbox rescaling, the corrective-pass routing
• TIER 0	Built, plausible, unmeasured	Every prompt-driven behaviour — motif consistency, symbol classification, register matching, the entire adaptation quality claim
• TIER 2	Would need research, not engineering	Cross-language emotional fidelity — comparing the shipped English against the actual source rather than against the anchor

27 entries are Tier 0. That is not a criticism of the tiering — it is the tiering working correctly. Prompt-driven behaviour genuinely cannot be verified by unit tests; you can test that a prompt contains an instruction, never that the model obeyed it well. The only instrument that converts Tier 0 into evidence is the blind benchmark.

What the benchmark would actually produce

It runs Castia against a single GPT prompt, a single Claude prompt, Google Translate, and published human translations on identical songs; normalises every output so typography can't identify a system; splits them into per-reviewer blinded packets; and reports with significance tests.

The `gpt_single` comparison is the one that decides the project's premise, because it holds the model constant and varies only the process. Everything Castia does beyond a single prompt — the anchor, the five philosophies, the specialists, the ledger, the verifier, the corrective pass — is justified if and only if that gap is real.

What doesn't work yet

Stated plainly, in rough order of how much they matter.

GAP	WHY IT MATTERS	DIFFICULTY
Nothing is measured	The core quality claim is unevidenced. Every other item on this list is secondary to it.	Low effort, high coordination — needs bilingual reviewers
Cross-panel speaker identity	Speaker labels are panel-local, so per-character voice consistency across a chapter cannot actually happen yet. The data to fix it is already emitted.	1–2 days, no new infrastructure
Cross-language fidelity verification	The deepest structural gap: the verifier checks the shipped English against the Translator's anchor, never against the actual source text. A Translator error propagates undetected through everything downstream.	Research — bilingual NLI or back-translation
Per-bubble comics adaptation	Each panel is adapted as one block of dialogue, not per speech bubble — which is why redraw needs per-region text typed by hand for multi-bubble panels.	Medium — schema and orchestration
Chorus with variation	Only exact-verbatim repeats are represented. A chorus that repeats with one line changed has no structural representation, so the repeat-preservation check can't see it.	Medium — new schema
Comics feature parity	No save, collections, share links, or async jobs. A long chapter can time out.	Low, but real product work
Typography matching	One bundled font for every redrawn line. The vision pass now captures lettering emphasis, but nothing consumes it yet.	Medium — the input already exists
No billing	No Stripe, no metering beyond quotas. Nothing can be charged for.	Low, well-understood
Genre calibration	Thresholds are global. What counts as enough rhyme differs sharply between a Hindi film couplet and a traditional Japanese lyric.	Blocked on corpus data

Four horizons

Beyond the immediate recommendation in §00.

Horizon 1 — Prove the premise

Run the benchmark. Recruit two or three bilingual reviewers. Publish the result whichever way it falls, because a null result is also worth knowing: if a single well-written prompt matches the room, the right response is to simplify aggressively and compete on the verifier and the workflow instead — both of which would still be genuinely differentiated.

Horizon 2 — Close the comics loop

Speaker reconciliation across panels, then per-bubble adaptation, then let the captured lettering emphasis actually drive typesetting. These three compound: once bubbles are individually adapted and attributed, redraw stops needing hand-typed per-region text and the whole chapter workflow becomes genuinely automatic rather than assisted.

Horizon 3 — Make the differentiator visible

The verifier is the most defensible thing in the project and it is currently invisible to users. Every adaptation already produces a full audit — ledger coverage, adaptation distance, which laws were checked, what the corrective pass changed. Surfacing that as a quality report would turn an internal mechanism into the reason a professional chooses this tool. Localisation teams buy defensibility; showing the audit is showing the product.

Horizon 4 — Earn the price

Only after the above. The paid tiers are currently described as what's being built toward, which is honest; a translation memory or glossary that persists decisions across chapters, and batch/API workflows, are the features that make a subscription rational for a team rather than a nicety for an individual.

ON THE ENTERPRISE-LOCALISATION CHECKLIST

Translation memory, CMS and Figma integrations, video dubbing with lip-sync — that list describes Smartling and Lokalise. It is a different product for a different buyer: a company managing thousands of strings, not a person adapting a song. Chasing it would mean abandoning the only thing Castia currently does better than anyone, which is caring about whether the line still *means* something.

What could sink it

- **The premise doesn't hold.** The frontier models improve fast, and a single prompt to a 2026-class model may already do most of what the room does. This is the existential one, it is cheap to test, and it has not been tested.
- **Cost per adaptation.** A song is Song DNA plus three-to-seven calls per section, plus a corrective pass. That is an order of magnitude more expensive than a single prompt. Without a demonstrated quality gap, it is an order of magnitude of pure loss.
- **The architecture is ahead of the evidence.** Three service boundaries exist for models that have never run. That work is not wasted — the fallbacks make it free to carry — but it is a standing invitation to keep building instead of validating.
- **Two products, one team.** Music and Webtoons share an engine but not a buyer, a workflow, or a competitive set. Splitting focus before either is proven is how both stay at eighty percent.
- **Rights and licensing.** Lyrics and comics are someone's copyrighted work. Adaptation for personal use is one posture; a commercial localisation service for publishers is another, and it eventually needs a real answer.

None of these are reasons to stop. They are reasons to sequence differently: the cheapest way to retire the largest risk is to run an experiment that already exists, on songs that are already in the repository, using a harness that was built for exactly this and has been sitting idle.

Castia – engineering & product dossier

Compiled from the working tree at commit f0c96eb · 145 commits · 828 tests